# MEMORANDUM


**To:** Charlie Refvem, Lecturer, Department of Mechanical Engineering, Cal Poly SLO
crefvem@calpoly.edu

**From:** Quinn Patterson
qpatters@calpoly.edu
Jake Rowen
jhrowen@calpoly.edu

**Team:** 23


**Date:** 2/21/26

**RE:** **Lab 0x04 Memo**

**Table of Contents**


1. Memo Body
2. Plots
3. Shared Variables
4. Task Profile
5. Program Functionality
6. Source Code


**Memo Body**

This week's task was to create a more robust user interface for our Romi, allowing us to change our motors proportional and integral gain, as well as setpoint. We can then run the motors and collect step response data from them. An auxiliary python script on the user's computer then can read these outputs and create step response plots from the data. In this user script, $K_P$, $K_I$, and setpoint values are chosen and sent to Romi. Romi receives these inputs and changes them accordingly. The script then will tell Romi to run a step response and collect the data it outputs, placing it in a csv file in a local location. It then uses this csv to generate a step response plot that can be optionally saved next to the csv. Our best response is pictured below in Figure 1. Our value climbed quickly and then maintained effectively throughout the response. However, we have a lot of steady state error we need to correct in the future as our setpoint was 400mm/s.

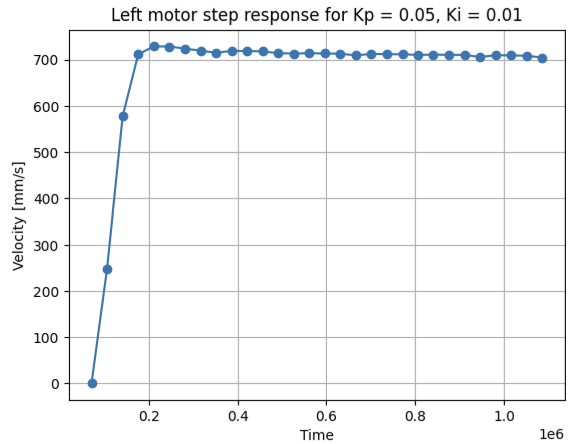



**Figure 1. Best Step Response**

Inside Romi, we implemented no new tasks but many new shares and queues to aid in creating a more robust user interface. We implemented individual queues to set each motors KP, KI, and setpoint as well as shares to tell the motors to run responses. This allows task_user to communicate with task_motor. We then have queues to save step response data coming out of our individual motors. This allows task_motor to communicate with task_user. The drawn-out structure for this can be viewed below in Figure 2. All the shares and queues we have implemented can be viewed below in the "Shared Variables" section of the memo in more detail.

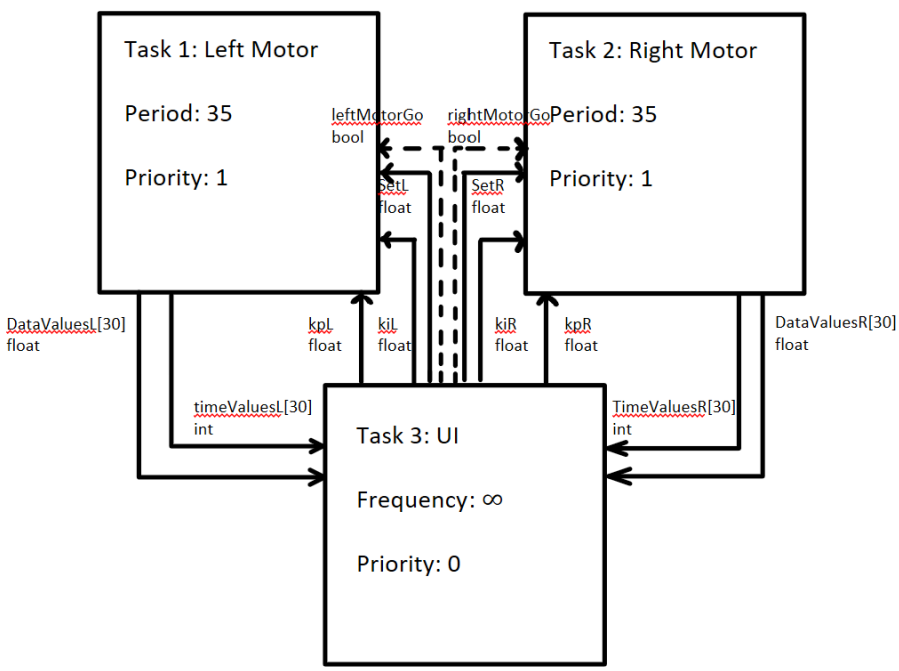



**Figure 2. Task Diagram**

Our left motor task and right motor task both have a priority of one and run with a period of 35 milliseconds. 35 milliseconds was found to be a proper time to get usable velocity data while not waiting too long to collect a new data point. Both are priority one because one motor

should not take priority over the other. Our user task has a priority of zero and runs with a period of zero. This way it does not get in the way of our motors and runs whenever they are not being used. The finite state machines for our user and motor tasks can be seen below in Figures 3 and 4.

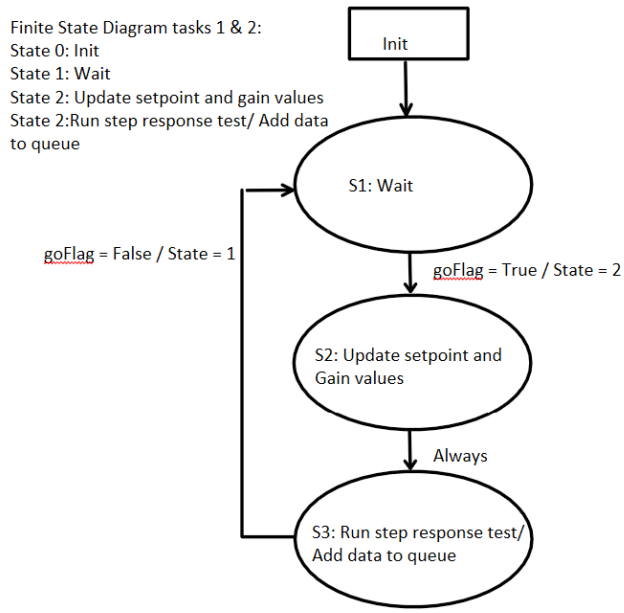



**Figure 3. task_user.py State Transition Diagram**

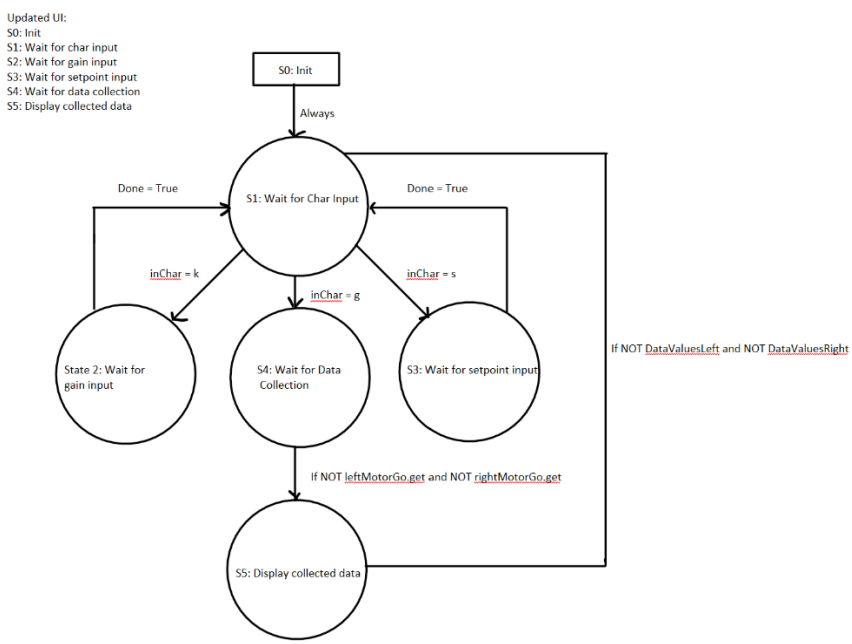



**Figure 4. task_motor.py State Transition Diagram**

**Plots**

Included below are three plots, all with a setpoint of 400mm/s, shown from worst to best response. Only the left motor response was shown because they look identical to the right motor.

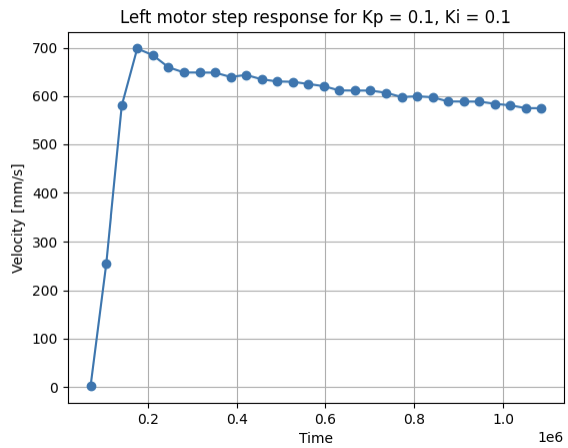



**Figure 5. Worst Response**

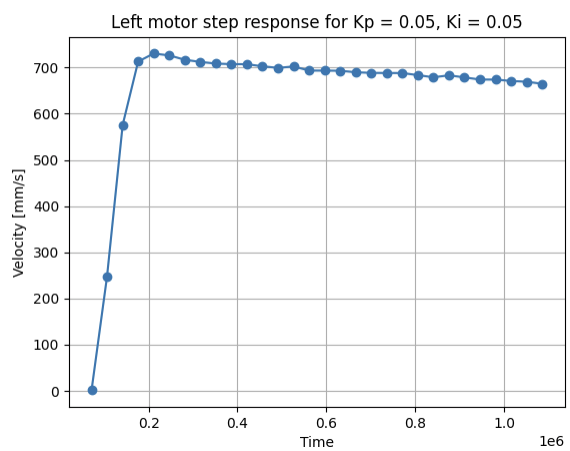



**Figure 6. Medium Response**

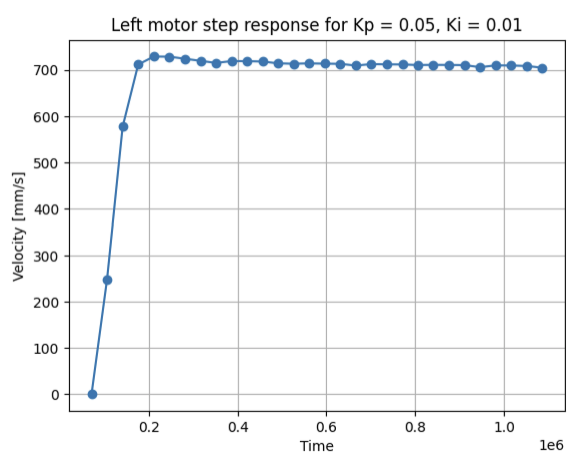



**Figure 7. Best Response**

**Shared Variables**

| Name | Type | Purpose |
|---|---|---|
| LeftMotorGo | Share "B" | Tells the left motor to run its step response |
| RightMotorGo | Share "B" | Tells the right motor to run its step response |
| Left_Motor_KP | Queue "f", size 1 | Tells left motor to change KP when queue populated by the user |
| Left_Motor_KI | Queue "f", size 1 | Tells left motor to change KI when queue populated by the user |
| Left_Motor_Setpoint | Queue "f", size 1 | Tells left motor to change setpoint when queue populated by user |
| Right_Motor_KP | Queue "f ", size 1 | Tells right motor to change KP when queue populated by the user |
| Right_Motor_KI | Queue "f", size 1 | Tells right motor to change KI when queue populated by the user |
| Right_Motor_Setpoint | Queue "f", size 1 | Tells right motor to change setpoint when queue populated by user |
| dataValuesleft | Queue "f", size 30 | Output queue for left motor to put velocity values of step response data points |
| timeValuesleft | Queue "f", size 30 | Output queue for left motor to put time values of step response data points |
| dataValuesright | Queue "f", size 30 | Output queue for right motor to put velocity values of step response data points |
| timeValuesright | Queue "f", size 30 | Output queue for right motor to put velocity values of step response data points |

**Table 1. Shared Variables and Their Function**

**Task Profile**

The table below showcases the results of running the task profile from cotask.py. It claims that User Int. Task never tuns even though we could provide user input but other than that it does not appear there was any interference or major blockages. The maximum time a task was late was Right Mot. Task with 2.44 ms.

| Task | PRI | PERIOD | RUNS | AVGDUR | MAXDUR | AVGLATE | MAXLATE |
|---|---|---|---|---|---|---|---|
| Left Mot. Task | 1 | 35.0 | 1489 | 0.139 | 1.462 | 0.267 | 1.295 |
| Right Mot. Task | 1 | 35.0 | 1489 | 0.147 | 1.480 | 0.227 | 2.440 |
| User Int. Task | 0 | 0.0 | 0 | 0 | 0 | 0 | 0 |

**Table 2. Task Profile Data**

## Program Functionality

Figures 8 and 9 below showcase that the code can run multiple times in a row, creates plots, then saves the data in a csv with the Kp, KI, and setpoint in the name.

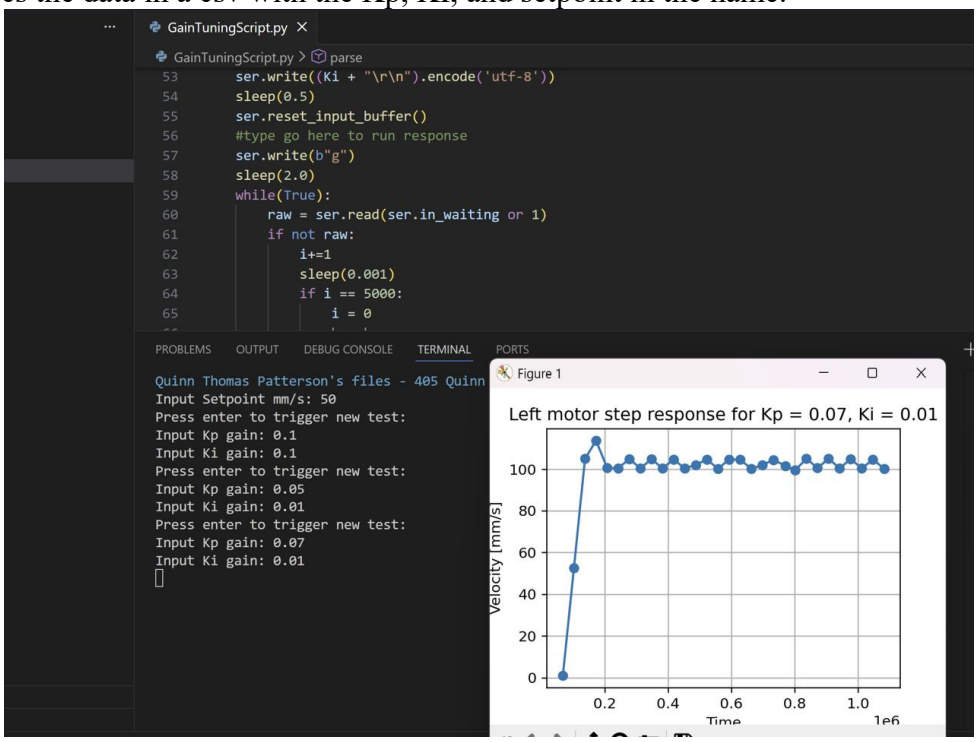
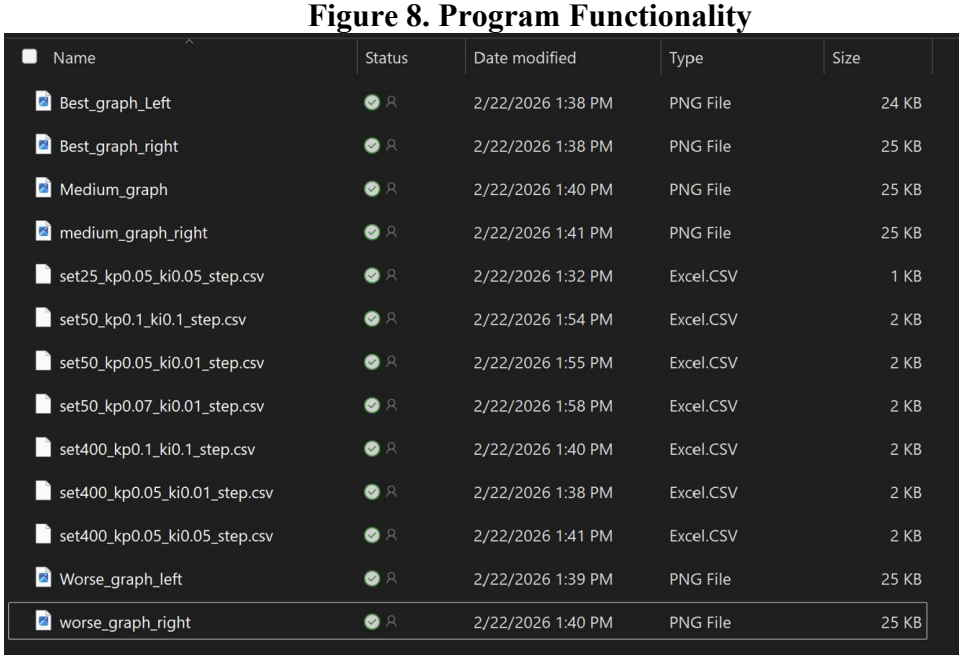



**Figure 8. Program Functionality**



| Name | Status | Date modified | Type | Size |
|---|---|---|---|---|
| Best_graph_Left | | 2/22/2026 1:38 PM | PNG File | 24 KB |
| Best_graph_right | | 2/22/2026 1:38 PM | PNG File | 25 KB |
| Medium_graph | | 2/22/2026 1:40 PM | PNG File | 25 KB |
| medium_graph_right | | 2/22/2026 1:41 PM | PNG File | 25 KB |
| set25_kp0.05_ki0.05_step.csv | | 2/22/2026 1:32 PM | Excel.CSV | 1 KB |
| set50_kp0.1_ki0.1_step.csv | | 2/22/2026 1:54 PM | Excel.CSV | 2 KB |
| set50_kp0.05_ki0.01_step.csv | | 2/22/2026 1:55 PM | Excel.CSV | 2 KB |
| set50_kp0.07_ki0.01_step.csv | | 2/22/2026 1:58 PM | Excel.CSV | 2 KB |
| set400_kp0.1_ki0.1_step.csv | | 2/22/2026 1:40 PM | Excel.CSV | 2 KB |
| set400_kp0.05_ki0.01_step.csv | | 2/22/2026 1:38 PM | Excel.CSV | 2 KB |
| set400_kp0.05_ki0.05_step.csv | | 2/22/2026 1:41 PM | Excel.CSV | 2 KB |
| Worse_graph_left | | 2/22/2026 1:39 PM | PNG File | 25 KB |
| worse_graph_right | | 2/22/2026 1:40 PM | PNG File | 25 KB |

**Figure 9. Saved CSV Data**

**Source Code**

1. cotask.py
    a. This file contains classes to run cooperatively scheduled tasks in a multitasking system.
2. cqueue.py
    a. Not sure but removing it breaks things. Everything in it says that it is a documentation for the real file written in C but Romi will not run without it.
3. encoder_class.py
    a. Initializes encoder as a class object and includes useful functions to update position, get current position, get current velocity, and zero itself.
4. GainTuningScript.py
    a. Python script that interacts with our Romi via bluetooth. Sends setpoint, Kp, Ki values and receives step response data. Saves data and resulting plots in external labelled files.
5. main.py
    a. Creates our motor, encoder, and timer objects. Initializes scheduler and our tasks to run inside of it.
6. motor_class.py
    a. Initializes our motors as class objects and includes useful functions to enable, disable, and set effort of motors.
7. Motor_control.py
    a. Initializes our PI control for each motor and includes functions to change KP and KI. Includes function to run control loop.
8. multichar_input.py
    a. Dead script used as a reference to look back on for character input function in task_user.py. Includes comments that we removed in task_user.py for readability of the code
9. task_motor.py
    a. Receives shares and queues from task_user.py to change Kp, KI, and Setpoint values as well as run motor step responses. Sends queues with step response data back to task_user.py. Uses a motor control object and encoder control object from each motor to run them properly.
10. task_share.py
    a. Allows tasks to share data without the risk of data corruption by interrupts
11. task_user.py
    a. Receives user input and prints step responses to the serial port.